

JavaScript Async Programming — Answers

setTimeout, Callbacks, Promises, and Simulated API/Product Searches

Task 1

Create an HTML file with a script tag. Display “Loading your feed...” after 2 seconds, then replace it with “Feed loaded!” after another 2 seconds.

```
<!DOCTYPE html>
<html>
<head>
  <title>Feed Loader</title>
</head>
<body>
  <p id="message"></p>

  <script>
    const message = document.getElementById("message");

    setTimeout(() => {
      message.textContent = "Loading your feed...";

      setTimeout(() => {
        message.textContent = "Feed loaded!";
      }, 2000);
    }, 2000);
  </script>
</body>
</html>
```

Result: The page is blank initially, shows “Loading your feed...” after 2 seconds, and shows “Feed loaded!” 2 seconds later.

Task 2

Create the same HTML feed-loading example using setTimeout.

```
<!DOCTYPE html>
<html>
<head>
  <title>Feed Loader</title>
</head>
<body>
  <p id="feedMessage"></p>

  <script>
    const feedMessage = document.getElementById("feedMessage");

    setTimeout(() => {
      feedMessage.textContent = "Loading your feed...";

      setTimeout(() => {
        feedMessage.textContent = "Feed loaded!";
      }, 2000);
    }, 2000);
  </script>
</body>
</html>
```

Result: “Loading your feed...” appears after 2 seconds, then “Feed loaded!” appears after another 2 seconds.

Task 3

Create an HTML file with a script tag that logs “Loading...” immediately and “Content loaded!” after 2 seconds.

```
<!DOCTYPE html>
```

```

<html>
<head>
  <title>Content Loader</title>
</head>
<body>

  <script>
    console.log("Loading...");

    setTimeout(() => {
      console.log("Content loaded!");
    }, 2000);
  </script>

</body>
</html>

```

Console: Loading... (immediately), then Content loaded! (after 2 seconds).

Task 4

Simulate a Zomato order status updater using callbacks. `placeOrder` takes a food item and callback, waits 1 second, then calls the callback.

```

function placeOrder(item, callback) {
  setTimeout(() => {
    callback(`Your ${item} is being prepared`);
  }, 1000);
}

placeOrder("Pizza", (message) => {
  console.log(message);
});

```

After 1 second: Your Pizza is being prepared

Task 5

Create the same callback-based Zomato order status updater and test it.

```

function placeOrder(item, callback) {
  setTimeout(() => {
    callback(`Your ${item} is being prepared`);
  }, 1000);
}

placeOrder("Burger", (message) => {
  console.log(message);
});

```

After 1 second: Your Burger is being prepared

Task 6

Write `fetchSongLyrics(song, callback)` to simulate fetching lyrics from a server using `setTimeout`, then call the callback with the lyrics string.

```

function fetchSongLyrics(song, callback) {
  setTimeout(() => {
    const lyrics = `Lyrics for "${song}": This is a sample lyrics response.`;
    callback(lyrics);
  }, 1000);
}

fetchSongLyrics("Perfect", (lyrics) => {
  console.log(lyrics);
});

```

After 1 second: Lyrics for "Perfect": This is a sample lyrics response.

Task 7

Build a Promise called `getFollowersCount` that resolves after 1 second with a random number between 1000 and 5000. Use `.then()` to display the result.

```
function getFollowersCount() {
  return new Promise((resolve) => {
    setTimeout(() => {
      const count = Math.floor(Math.random() * 4001) + 1000;
      resolve(count);
    }, 1000);
  });
}

getFollowersCount().then((count) => {
  console.log(`You have ${count} new followers`);
});
```

Example output: You have 3274 new followers. The number changes randomly each run.

Task 8

Create the same `getFollowersCount` Promise and use `.then()` to display the follower message.

```
function getFollowersCount() {
  return new Promise((resolve) => {
    setTimeout(() => {
      const count = Math.floor(Math.random() * 4001) + 1000;
      resolve(count);
    }, 1000);
  });
}

getFollowersCount().then((count) => {
  console.log(`You have ${count} new followers`);
});
```

Example output: You have 4812 new followers. The number is random.

Task 9

Write `getOrderStatus(orderId)` returning a Promise. After 1.5 seconds, resolve for even IDs and reject for odd IDs. Handle both with `.then()` and `.catch()`.

```
function getOrderStatus(orderId) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (orderId % 2 === 0) {
        resolve("Order delivered");
      } else {
        reject("Order not found");
      }
    }, 1500);
  });
}

getOrderStatus(102)
  .then((message) => {
    console.log(message);
  })
  .catch((error) => {
    console.log(error);
  });

getOrderStatus(103)
  .then((message) => {
    console.log(message);
  })
  .catch((error) => {
    console.log(error);
  });
```

After 1.5 seconds: Order delivered for 102; Order not found for 103.

Task 10

Rewrite a Flipkart-style product search using both a callback version and a Promise version. Wait 1 second and return products whose names include the search term.

```
const products = [
  { name: "iPhone 15", price: 70000 },
  { name: "Samsung Galaxy S24", price: 65000 },
  { name: "OnePlus 12", price: 55000 },
  { name: "iPhone 14", price: 60000 }
];

// Callback version
function searchProductsCallback(products, searchTerm, callback) {
  setTimeout(() => {
    const results = products.filter(product =>
      product.name.toLowerCase().includes(searchTerm.toLowerCase())
    );

    callback(results);
  }, 1000);
}

// Promise version
function searchProductsPromise(products, searchTerm) {
  return new Promise((resolve) => {
    setTimeout(() => {
      const results = products.filter(product =>
        product.name.toLowerCase().includes(searchTerm.toLowerCase())
      );

      resolve(results);
    }, 1000);
  });
}

// Test callback version
searchProductsCallback(products, "iphone", (results) => {
  console.log("Callback results:", results);
});

// Test Promise version
searchProductsPromise(products, "samsung")
  .then((results) => {
    console.log("Promise results:", results);
  });
```

Callback search “iphone” returns iPhone 15 and iPhone 14. Promise search “samsung” returns Samsung Galaxy S24. Both wait 1 second.

Task 11

Create another implementation of the same Flipkart-style search using callback and Promise approaches, each with a 1-second simulated delay.

```
const products = [
  { name: "Laptop", price: 60000 },
  { name: "Wireless Mouse", price: 1200 },
  { name: "Laptop Bag", price: 1800 },
  { name: "Keyboard", price: 2500 }
];

// Callback version
function searchProductsCallback(products, searchTerm, callback) {
  setTimeout(() => {
    const results = products.filter(product =>
      product.name.toLowerCase().includes(searchTerm.toLowerCase())
    );

    callback(results);
  }, 1000);
}

// Promise version
function searchProductsPromise(products, searchTerm) {
```

```

return new Promise((resolve) => {
  setTimeout(() => {
    const results = products.filter(product =>
      product.name.toLowerCase().includes(searchTerm.toLowerCase())
    );

    resolve(results);
  }, 1000);
});
}

searchProductsCallback(products, "laptop", (results) => {
  console.log("Callback results:", results);
});

searchProductsPromise(products, "mouse")
  .then((results) => {
    console.log("Promise results:", results);
  });

```

Callback search "laptop" returns Laptop and Laptop Bag. Promise search "mouse" returns Wireless Mouse.

Task 12

Refactor the callback-based payment code to use a Promise. The Promise should resolve after 1 second and log "Payment successful!".

```

function processPayment() {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Payment successful!");
      resolve();
    }, 1000);
  });
}

processPayment().then(() => {
  console.log("Payment process completed.");
});

```

After 1 second: Payment successful! Then: Payment process completed.